N-Gram Language Models

CS 485, Spring 2026
Applications of Natural Language Processing

Katrin Erk
College of Information and Computer Sciences
University of Massachusetts Amherst

[slides from JM3, Mohit Iyyer, Brendan O'Connor]

goal: assign probability to a piece of text

- Goal #1: assign probability to a piece of text
  - translation:
    - P(i flew to the movies) <<<<< P(i went to the movies)
  - speech recognition:
    - P(i saw a van) >>>>> P(eyes awe of an)
  - text classification (NB):
    - P(i am so mad!! | *[author is happy]*) <
      P(i am so mad!! | *[author is not happy]*)
- Goal #2: generate text from this defined probability distribution

Probabilistic language modeling

- Goal: compute the probability of a sentence or of a sequence of words:
  $P(W) = P(w_1 w_2 w_3 \ldots w_n)$

- Related task: compute the probability of an upcoming word:
  $P(w_4 \mid w_1 w_2 w_3)$

- A model that computes either of these, either P(W) or $P(w_4 \mid w_1 w_2 w_3)$, is called a l**anguage model or LM.**

How would you compute P(W)?

- P(its, water, is, so, transparent, that)

How would you compute P(W)?

- P(its, water, is, so, transparent, that)
- Can we break up this long joint probability?
- Intuition: Let’s rely on the Chain Rule of Probability

Reminder: the Chain Rule

- How are conditional probabilities defined?
  $P(A|B) = \ldots$

Reminder: the Chain Rule

- How are conditional probabilities defined?
  $P(A \mid B) = \dfrac{P(A, B)}{P(B)}$ Rewriting: $P(A, B) = P(A \mid B)P(B)$

- More variables:
  $P(A, B, C, D) = P(A)P(B \mid A)P(C \mid A, B)P(D \mid A, B, C)$

- The Chain Rule general:
  $P(x_1, x_2, x_3 \ldots, x_n) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2) \ldots P(x_n \mid x_1, \ldots, x_{n-1})$

Applying the chain rule to computing P(W)

- $P(W) = P(w_1 w_2 \ldots w_n) = P(w_1)P(w_2 | w_1)\ldots P(w_n | w_1 w_2 \ldots w_{n-1})$

- P(its water is so transparent that) =
  P(its)
  P(water|its)
  P((is | its water)
  P(so | its water is)
  P(transparent | its water is so)
  P(that | its water is so transparent)

- How do we compute this? Let's look at one step.

Chain rule: Let’s try one step

- How to compute P(that | its water is so transparent)?

Chain rule: Let's try one step

- How to compute P(that | its water is so transparent)?
- Could we do relative frequency, like with Naive Bayes?
- P(that | its water is so transparent) ≈

  Count(its water is so transparent that)

  ——————————————————————-

  Count(its water is so transparent _)
- What is the problem with that?

Chain rule: Let's try one step

- P(that | its water is so transparent) ≈
  Count(its water is so transparent that)
  ——————————————————————-
  Count(its water is so transparent _)
- What is the problem with that?
- Count(its water is so transparent that): much too rare, will be zero!

Let’s make matters simpler

- Markov assumption:

  $P(\text{word}_n \mid \text{very many words}) \approx P(\text{word}_n \mid \text{word}_{n-1})$
- or maybe

  $P(\text{word}_n \mid \text{very many words}) \approx P(\text{word}_n \mid \text{word}_{n-1}\ \text{word}_{n-2})$
- One word context:

  P(that | its water is so transparent) ≈ P(that | transparent)
- 2 words context:

  P(that | its water is so transparent) ≈ P(that | so transparent)

Andrei Markov (1856~1922)

Markov assumption

- One word of context:
  $$P(w_1 \ldots w_n) = \prod_i P(w_i \mid w_{i-1})$$
- analogously for longer contexts
- In other words, we approximate each component:
  $$P(w_i \mid w_1 \ldots w_{i-1}) \approx P(w_i \mid w_{i-1})$$

Simplest case: unigram model

- $P(w_1 \ldots w_n) = P(w_1) \cdot \ldots \cdot P(w_n)$

- Generate one word at a time, sampling based on corpus frequency (more frequent words get generated more often)

- Some automatically generated “sentences” from a unigram model:
  - `fifth an of futures the an incorporated a a the inflation most dollars quarter in is mass`
  - `thrift did eighty said hard 'm july bullish`
  - `that or limited the`

Approximating Shakespeare

1-gram

-To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

--Hill he late speaks; or! a more to leg less first you enter

2-gram

-Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

-What means, sir. I confess she? then all sorts, he is trim, captain.

3-gram

-Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

-This shall forbid it should be branded, if renown made it empty.

4-gram

-King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

-It cannot be but so.

Approximating Shakespeare

1-gram

-To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

--Hill he late speaks; or! a more to leg less first you enter

2-gram

-Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

-What means, sir. I confess she? then all sorts, he is trim, captain.

3-gram

-Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

-This shall forbid it should be branded, if renown made it empty.

4-gram

-King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

-It cannot be but so.

The 2-grams are surprisingly good, right?
What would 10-grams look like?

N-gram models

- Can extend n-grams to higher n...
- N-gram models are surprisingly useful; state of the art for >50 years!
- But this is an insufficient model of language
  - Long-distance dependencies
  - Language is compositional
  - Generalize effects beyond distinct words

How do we estimate bigram probabilities?

How do we estimate bigram probabilities?

- Maximum Likelihood Estimation

How do we estimate bigram probabilities?

- Maximum Likelihood Estimation
- Relative frequencies based on empirical counts in a training set

$$P(w_i \mid w_{i-1}) \approx \frac{count(w_{i-1}, w_i)}{count(w_{i-1}, _)} = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

An example

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

What is…

P(I | <s>)

P(</s> | Sam)

P(Sam | <s>)

P(not | do)

Note: <s>, </s>
What are they good for?

$$P(w_i \mid w_{i-1}) \approx \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

An example

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

What is…

P(I | <s>) = 2/3

P(</s> | Sam) = 1/2

P(Sam | <s>) = 1/3

P(not | do) = 1

Note: <s>, </s>
What are they good for?

$$P(w_i \mid w_{i-1}) \approx \frac{count(w_{i-1}, w_i)}{count(w_{i-1}, _)}$$

Demo, and exercise

- Demo: Bigram language model
- Exercise:
  The demo comes with 3 files:
  Alice in Wonderland (alice.txt),
  Darwin's Origin of Species (species.txt), and
  Austen's Emma (emma.txt)

  Choose one file F OTHER THAN alice.txt,
  and rerun the whole notebook with F. (If tF is too big, stop reading after the first N lines.) Submit:
  - One paragraph of text (min 50 words) generated from a language model trained on F.
  - One word OTHER THAN "said", and its next-word probabilities (as in box 7) according to the language model trained on F
  - For two of the next-word probabilities, a comment on whether they reflect facts about syntax, semantics, the world, or something else

A bigger example

Berkeley Restaurant Project sentences

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i'm looking for a good place to eat breakfast
- when is caffe venezia open during the day

Raw bigram counts from a large corpus

Out of 9222 sentences

	I	want	to	eat	Chinese	food	lunch	spend
I	5	827	`0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
Chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Raw bigram probabilities

- Normalize by unigram counts: $P(w_i \mid w_{i-1}) \approx \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$

I	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Result:

	I	want	to	eat	Chinese	food	lunch	spend
I	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
Chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bigram estimate of sentence probabilities

- P(<s> I want english food </s>) =
  P(I | <s>)
  x P(want | I)
  x P(english | want)
  x P(food | english)
  x P(</s> | food)
  = 0.000031

- These probabilities get tiny when we have longer inputs with more infrequent words. How can we get around this?

What kinds of knowledge?

- P(english | want) = .0011 **about the world**
- P(chinese | want = .0065
- P(to | want) = .66 **grammar: infinitives**
- P(eat | to) = .28
- P(food | to) = 0 **what is this?**
- P(want | spend) = 0 **grammar**
- P(i | <s>) = .25

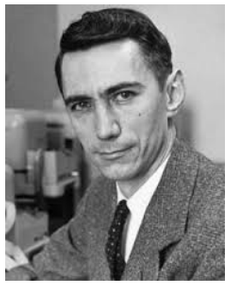
Evaluation

- How can we evaluate how good a language model is?

Evaluation

- How can we evaluate how good a language model is?
- Does the model give higher probability to real text?
  - ...in the test set (*why?*)

Evaluation



Claude Shannon
(1916~2001)

Does the model give higher probability to real text?

Intuition of Perplexity:

- The Shannon game:
  How well can we predict the next word?
  I always order pizza with cheese and ___
  The 33rd President of the US was ___
  I saw a ___
- Unigrams are terrible at this game. (Why?)
- A better model of a text
  is one which assigns a higher probability to the word that actually occurs
- Compute per word log likelihood: M words, m test sentences s_i

mushrooms 01
pepperoni 0.1
anchovies 001
..
fried rice 0.0001
…
and 1e-100

$$l = \frac{1}{M} \sum_{i=1}^{m} \log p(s_i)$$

Perplexity

- The best language model is one that best predicts an unseen test set — gives the highest P(sentence)
- Perplexity is the inverse probability of the test set, normalized by the number of words
- Minimizing perplexity is the same as maximizing probability

Perplexity

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

Chain rule: $PP(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_1 \ldots w_{i-1})}}$

For bigrams: $PP(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_{i-1})}}$

Lower perplexity = better model

- Training 38 million words, test 1.5 million words, Wall Street Journal

N-gram order	Unigram	Bigram	Trigram
Perplexity	962	170	109

n-gram sparsity: Shakespeare corpus

- N = 884,647 tokens, Vocab size V= 29,066
- Shakespeare produced 300,000 bigram types out of V^2=844 million possible bigrams
- So 99.96% of the possible bigrams were never seen: They have zero entries in the table
- Quadrigrams are worse: What is coming out looks like Shakespeare because it is Shakespeare

Sample zero's

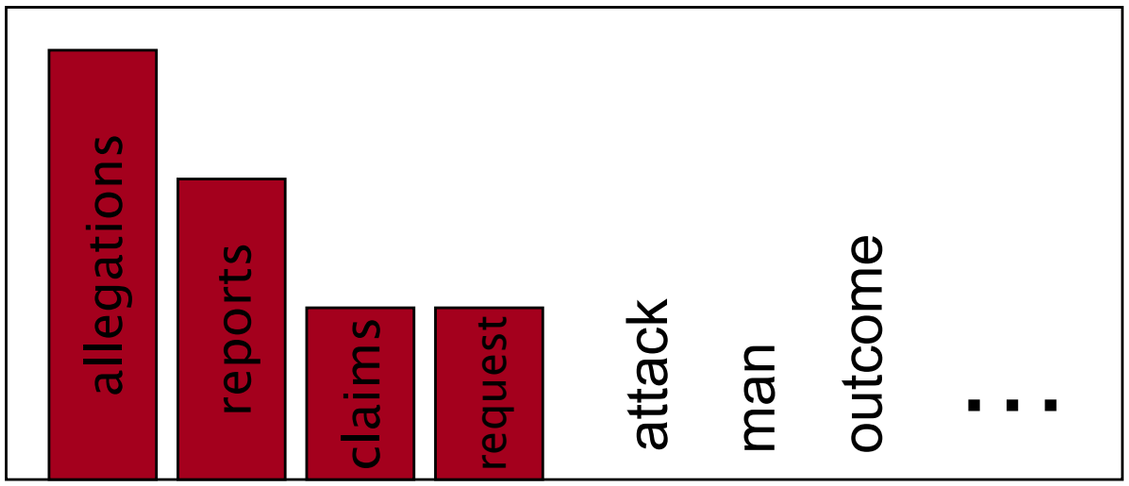
- Training set:
  … denied the allegations
  … denied the reports
  .. denied the claims
  … denied the request
- Test set:
  … denied the offer
  … denied the loan
- P(offer | denied the) = 0

- How does this affect perplexity?

The intuition of smoothing (from Dan Klein)

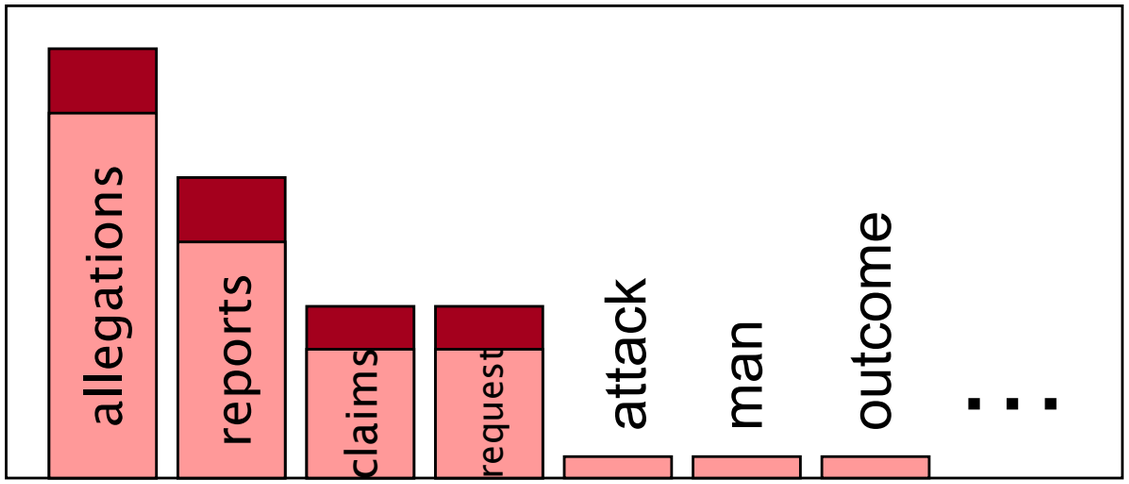
- When we have sparse statistics:

  P(w | denied the)
  3 allegations
  2 reports
  1 claims
  1 request
  7 total



- Steal probability mass to generalize better

  P(w | denied the)
  2.5 allegations
  1.5 reports
  0.5 claims
  0.5 request
  2 other
  7 total



Add-one estimation (again!)

- Also called Laplace smoothing
- Pretend we saw each word one more time than we did
- Just add one to all the counts
- Need to adapt the denominator so all weights sum up to one: add sum of additional counts to the denominator
- MLE estimate: $P_{MLE}(w_i \mid w_{i-1}) = \dfrac{c(w_{i-1}, w_i)}{c(w_{i-1})}$
- Add-one estimate: $P_{add1}(w_i \mid w_{i-1}) = \dfrac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$

Berkeley restaurant corpus: Laplace smoothed bigrams

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Berkeley restaurant corpus: Laplace smoothed bigrams

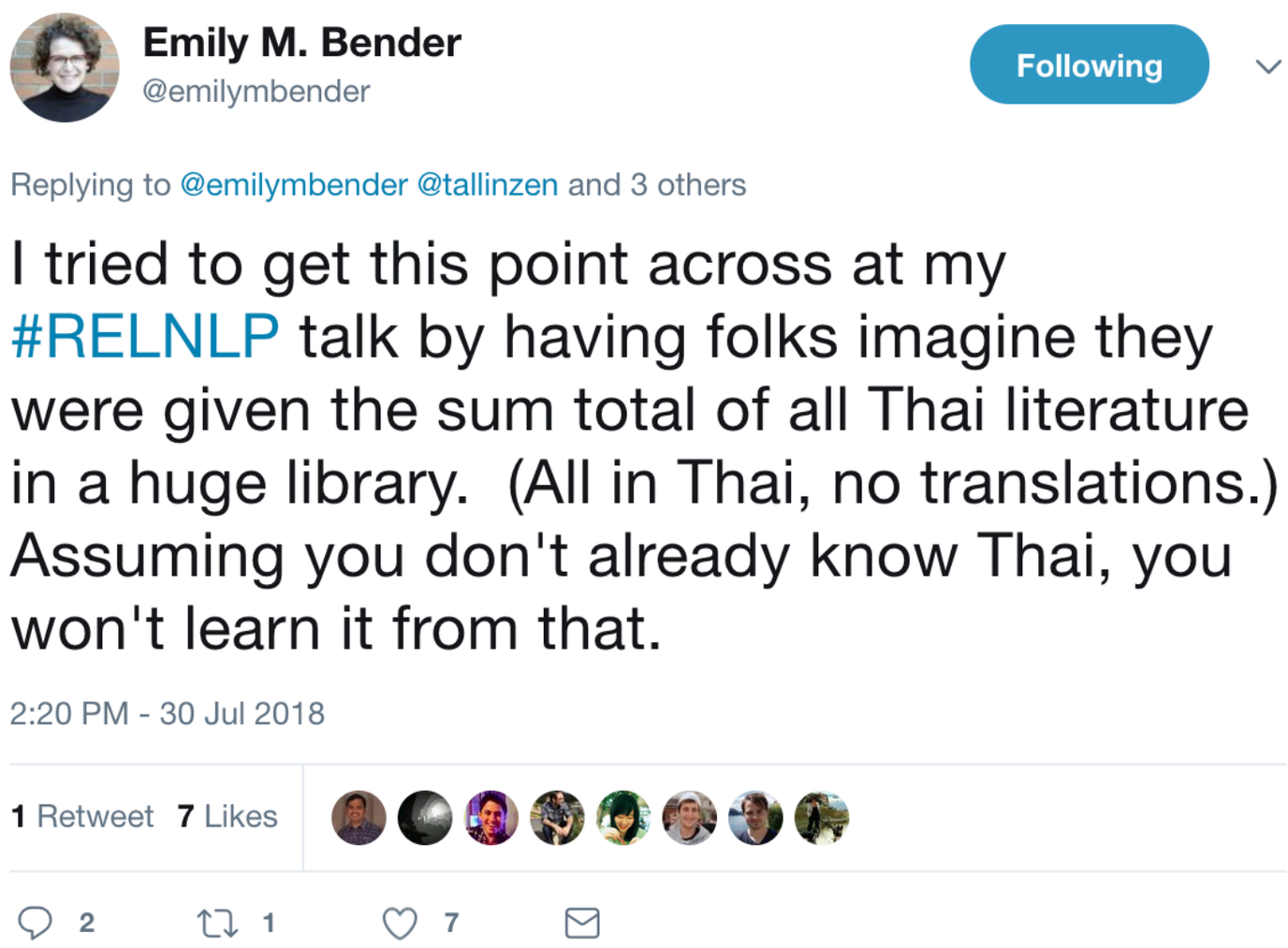
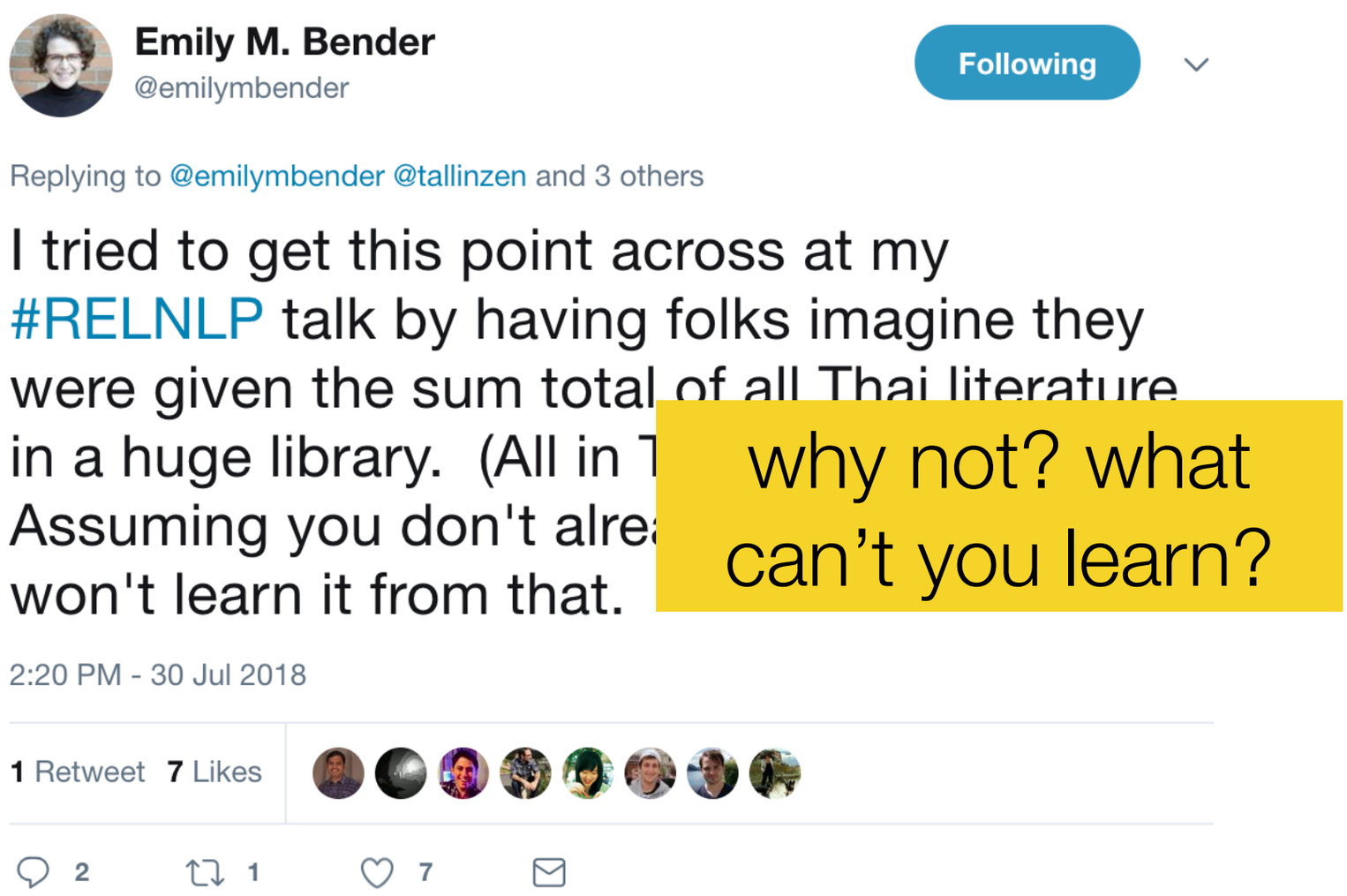
$$P_{add1}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Add-1 estimation is a blunt instrument

- Gives over too much probability mass to unseen events
- There are better methods
- But it's good to understand this simple method in order to understand the idea of smoothing

Aside: how far can you go with a model that only has access to text?



Aside: how far can you go with a model that only has access to text?